

MapReduce

Large-Scale Data Processing

Hoa Vu
CS 663

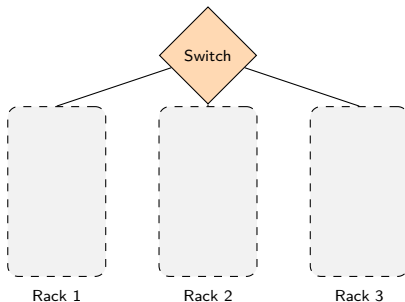
SDSU

April 6, 2026

The Big-Data Problem

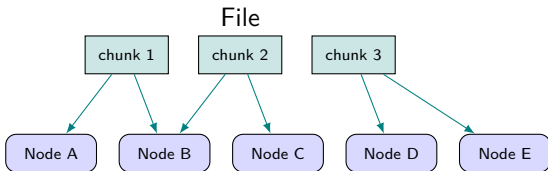
- ▶ Modern datasets: trillions of Web pages, social-network graphs, sensor logs.
- ▶ A single machine cannot store or process them in reasonable time.
- ▶ **Solution:** spread the work across *thousands* of commodity machines.
- ▶ Need a programming model that is:
 - ▶ simple to use,
 - ▶ automatically parallel,
 - ▶ fault-tolerant.

Cluster Architecture



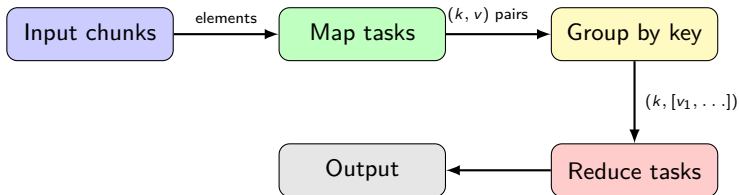
- ▶ Nodes on a rack: connected by gigabit Ethernet.
- ▶ Racks: interconnected by a high-speed switch.
- ▶ Hardware **failures are normal** and must be handled automatically.

Distributed File System (DFS)



- Files split into **64 MB chunks**, each replicated $\times 3$.
- Replicas on *different* racks $\Rightarrow$ survives rack failure.
- Examples: GFS (Google), HDFS (Hadoop/Apache).

MapReduce Overview



1. **Map**: user function turns each input element into (k, v) pairs.
2. **Shuffle**: system groups all values by key.
3. **Reduce**: user function aggregates each key's value list.

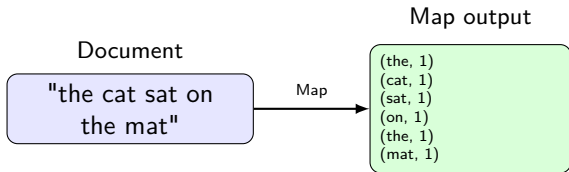
Key–Value Pairs

- ▶ The **Map** function outputs: $(key, value)$
- ▶ Keys are *not* unique — many pairs can share the same key.
- ▶ The system guarantees: all pairs with the same key reach *one* Reduce task.

- ▶ The **Reduce** function receives: $(key, [v_1, v_2, \dots, v_n])$
- ▶ It produces zero or more output pairs.

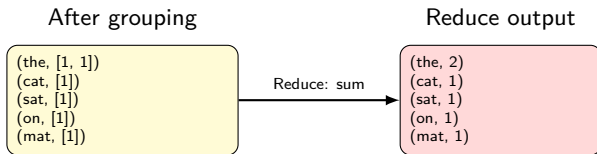
Example: Word Count

Input: a corpus of documents. **Goal:** count occurrences of each word.



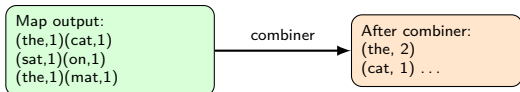
Example: Word Count (cont.)

After **grouping** by key:

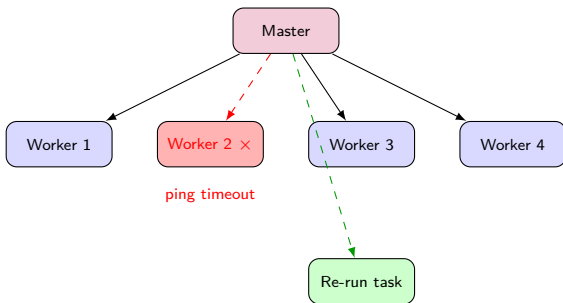


Combiners: Early Aggregation

- ▶ If **Reduce** is *associative and commutative*, we can apply it *locally* inside each Map task.
- ▶ This **combiner** reduces the data sent over the network.



Handling Failures

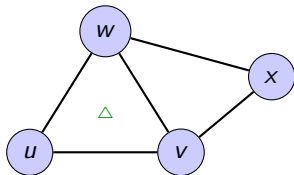


- **Master** pings workers periodically.
- Failed Map/Reduce tasks are **restarted** on another worker.
- Only Master failure requires restarting the whole job.

Triangle Counting

Goal: count triangles (u, v, w) in a large graph.

- ▶ A **triangle**: three nodes mutually connected.
- ▶ Used in: clustering coefficients, fraud detection, social-network analysis.



Triangle Counting

First round: for each edge (u, v) , emit two pairs

- ▶ $key = u, value = (u, v)$
- ▶ $key = v, value = (v, u)$

The reducer for node v receives all neighbors of v and emits pairs of neighbors as **wedges** back to the mappers.

For example, reducer for v receives $(v, a), (v, b), (v, c)$ it will emit wedges:

$wedge(a, b), wedge(a, c), wedge(b, c)$

edges:

$edge(v, a), edge(v, b), edge(v, c)$

Triangle Counting

Second round:

for each $wedge(u, w)$, emit key $\{u, w\}$.

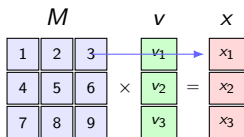
for each $edge(u, w)$, emit key $\{u, w\}$.

The reducer for key $\{u, w\}$ receives all wedges and edges between u and w . If both an edge and c wedges are present, then add a triangle to the count. Since each triangle is counted 6 times (once per node, twice per orientation), divide the total count by 6 at the end.

Matrix-Vector Multiplication

Compute $x = Mv$ where M is $n \times n$, $x_i = \sum_j m_{ij}v_j$.

- **Map:** from element m_{ij} emit $(i, m_{ij}v_j)$.
- **Reduce:** for key i , sum all values $\Rightarrow x_i$.

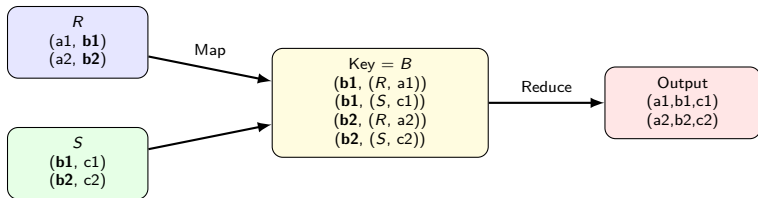


Map emits $(i, m_{ij}v_j)$; Reduce sums by i

Works at web scale: $n \sim 10^{12}$ (sparse Web-link matrix).

Natural Join Illustration

Join $R(A, B)$ and $S(B, C)$ on shared attribute B .

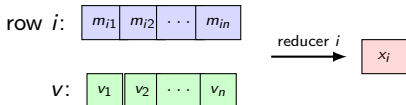


Communication Cost Model

- ▶ In MapReduce, computation is cheap; **data movement is the bottleneck**.
- ▶ **Communication cost** = total size of inputs to all Reduce tasks.
- ▶ **Reducer size** q = max inputs any one reducer receives.
- ▶ Trade-off: larger $q \Rightarrow$ fewer reducers $\Rightarrow$ less parallelism.

Communication Cost: Matrix-Vector Multiply

Multiply $n \times n$ matrix M by vector v . Each reducer computes one output $x_i = \sum_j m_{ij} v_j$.



Each reducer for output x_i receives:

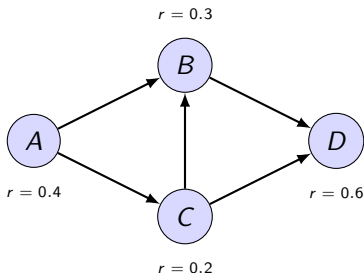
- ▶ n elements from row i of M
- ▶ n elements of v (entire vector)

$$q = 2n, \quad \text{total communication} = n \cdot 2n = 2n^2$$

PageRank: Idea

Goal: rank Web pages by importance.

A page is important if *important* pages link to it.

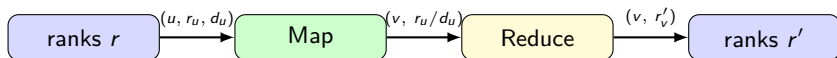


Each page u with rank r_u and out-degree d_u sends r_u/d_u to each neighbor.

PageRank: One Iteration via MapReduce

$$r'_v = \sum_{u \rightarrow v} \frac{r_u}{d_u}$$

- **Map:** for each page u with rank r_u and out-links $u \rightarrow v_1, \dots, v_{d_u}$: emit $(v_j, r_u/d_u)$ for each j .
- **Reduce:** for each page v , sum all received contributions $\Rightarrow r'_v$.



Repeat until convergence. Each iteration = one MapReduce job.